

SUBROTINAS E MODULARIZAÇÃO

LINGUAGEM DE PROGRAMAÇÃO

TALITA BERBEL

talita.berbel@fatec.sp.gov.br

Fatec
Sorocaba
José Crespo Gonzales

CP
Centro
Paula Souza

AGENDA

- Introdução
- Subrotina
- Tipos de Subrotinas
- Passagem de Parâmetros
- Chamada por valor
- Chamada por referência
- Ponteiros para Vetores e Ponteiros para Structs
- Modularização
- Conclusões e Dúvidas



INTRODUÇÃO

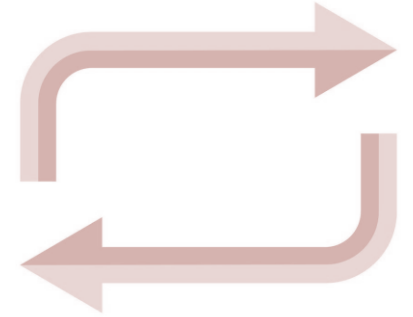
- A medida que os programas ganham mais possibilidade de fluxos, estruturas e também da quantidade de dados, faz-se necessário ações para organizar e encadear as instruções de um programa.
- Estudos desde os **tipos primitivos de dados**, estruturas de **decisão**, **seleção**, **repetição**, o uso de **vetores** e **matrizes** para dados do mesmo tipo, como **strings**, e **structs** para agrupar tipos de dados diferentes sobre um único nome, faz com que seja necessário **modularizar** partes do programa e criar **rotinas específicas** para cada parte lógica do programa.

SUBROTINA

- **Subrotina** é um trecho de programa que possui um nome.
- Para que as instruções de uma subrotina sejam executadas, basta invocar o seu nome, conforme as especificações da linguagem utilizada.
- Todas as linguagens comerciais suportam a construção e a utilização de subrotinas.



SUBROTINA



- A criação de sub-rotinas é recomendada nos casos de:
- **Repetição de código:** trechos de programa que se repetem com alguma frequência dentro da lógica. O código pode se repetir de forma exatamente igual em vários pontos ou pode possuir pequenas variações conforme o contexto em que é executado.
- **Identificação de hierarquias na lógica:** colocar em subrotinas os trechos de programa que implementam tarefas secundárias ou mais específicas, de maneira a “enxugar” a lógica dos módulos principais dos detalhes de implementação de certas operações.

BENEFÍCIOS DO USO DE SUBROTINAS

Vantagem	Descrição
Reutilização de código	Trechos repetidos são escritos uma única vez em subrotinas, aumentando a produtividade do programador.
Facilidade de manutenção	Alterações em uma subrotina afetam todos os lugares onde é chamada, facilitando atualizações.
Padronização de programas	Centralizar tarefas garante comportamento uniforme em todos os programas, evitando inconsistências.
Aumento da clareza	Programas ficam mais claros devido a estruturas menores, nomes mnemônicos e hierarquia visível das rotinas.
Diminuição do tamanho do código	Reduz o tamanho geral proporcionada principalmente pela reutilização de código.

TIPOS DE SUBROTINA

- Existem dois tipos de subrotinas:
- **Tipo Funções** ou ainda as **subrotinas que retornam valor** – utilizada quando a função precisa retornar um valor char, int, float ou double para a função que a referenciou.
 - Necessariamente deve-se ter o comando `return` na definição da função e função finaliza sua execução quando o `return` é encontrado.
- **Tipo Procedimento** ou ainda as **subrotinas que não retornam valor** – utilizada quando a função não precisa retornar valor algum para a função que a referenciou.
 - A função finaliza a sua execução quando finalizar o último comando dentro da função

TIPOS DE SUBROTINA

- **Tipo Funções** ou ainda as **subrotinas que retornam valor** – calcular o IMC.

```
float IMC(float Peso, float Altura){  
    return Peso / (Altura * Altura);  
}
```

- **Tipo Procedimento** ou ainda as **subrotinas que não retornam valor** – converter string em maiúscula.

```
void Maiusculas(char *texto) {  
    int cont;  
    for (cont = 0; texto[cont] != '\0'; cont++)  
        texto[cont] = toupper(texto[cont]);  
}
```

PASSAGEM DE ARGUMENTOS



- Nas chamadas das **funções** os argumentos devem ser compatíveis com os parâmetros em quantidade e tipo de dados.
- Exemplo:

```
resultado = IMC (valor_kg, altura_m); //chamada da função - argumentos
```

```
float IMC(float Peso, float Altura); //função - parâmetros
```

Two red arrows point from the parameters `valor_kg` and `altura_m` in the function call above to the parameters `Peso` and `Altura` in the function definition below.

- Os **argumentos** são os valores reais, `valor_kg` e `altura_m`.
- Os **parâmetros** são os guardadores de lugar para um argumento, sempre irão ter o tipo e o nome dos dados, `float Peso` e `float Altura`.

PASSAGEM DE ARGUMENTOS

- Em C existem dois tipos de passagem de argumentos para funções: chamada por valor e chamada por referência.
- A **chamada por valor** copia o valor da variável para a função, impedindo que a função original seja alterada, enquanto a **chamada por referência** passa o endereço da variável, permitindo que a função modifique o valor original.

CHAMADA POR VALOR

- Na **chamada por valor** é criada uma cópia do valor da variável original para ser usada dentro da função.
- Logo qualquer alteração feita na cópia dentro da função não afeta a variável original no escopo de quem a chamou.
- A chamada por valor é o método padrão em C e é útil quando é necessário usar o valor de uma variável sem a intenção de alterá-la.
- Exemplo:

```
void mostra(int x);  
  
int main(){  
    int num=10;  
    mostra(num);  
}
```

```
void mostra(int x){  
    printf("Valor copiado: %i\n",x);  
}
```

CHAMADA POR VALOR

- Exemplo:

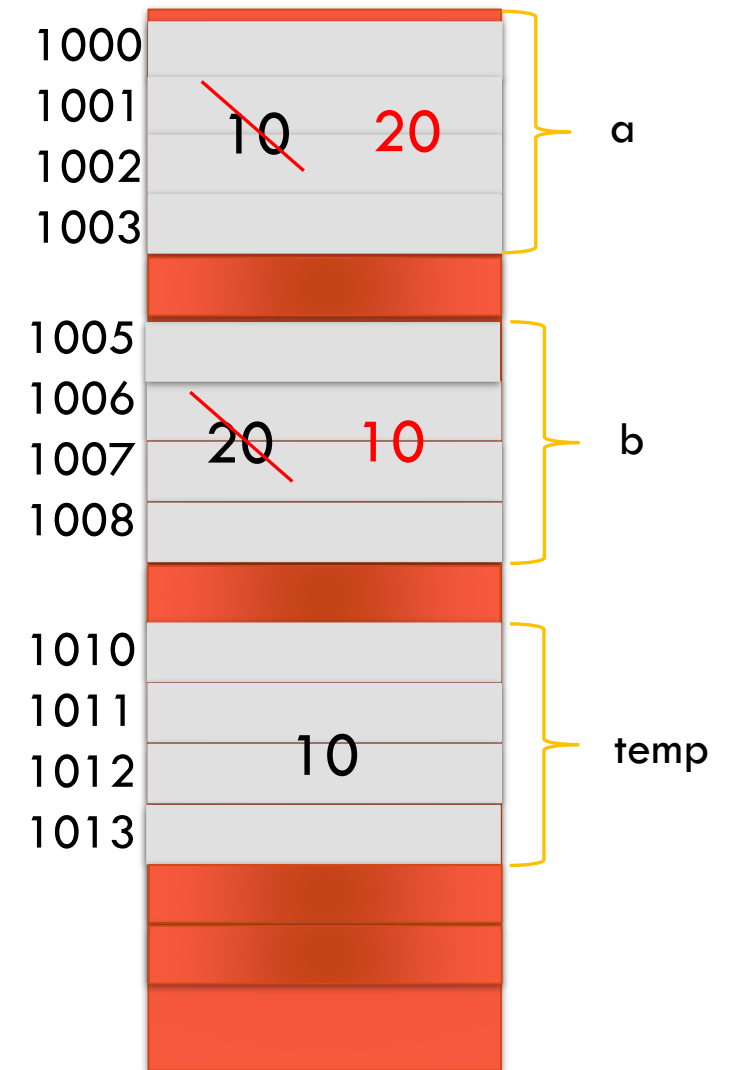
```
int main( )  
{  
    int a, b, temp;  
    a=10, b=20;  
    temp = a;  
    a = b;  
    b = temp;  
}
```

Entrada:

a=10 e b=20

Após a troca

a=20 e b=10



CHAMADA POR VALOR

- Exemplo:

```
int main( )
{
    int  a, b;
    a=10, b=20;
    troca(a , b);
}

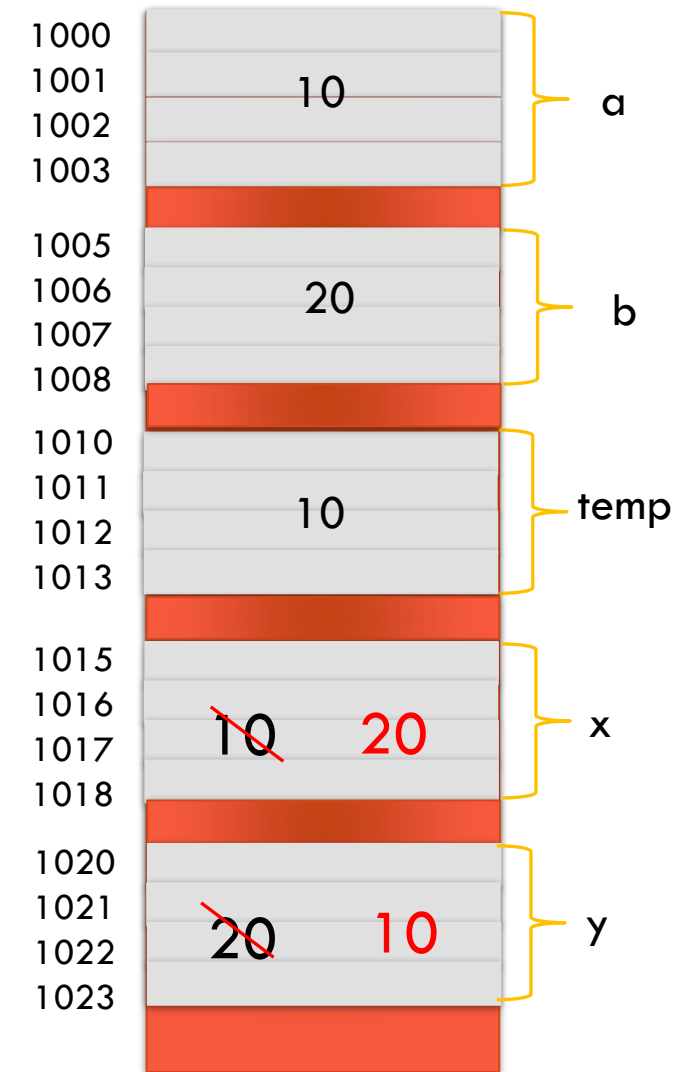
void  troca(int x, int y)
{
    int  temp;
    temp = x;
    x = y;
    y = temp;
}
```

Entrada:

a=10 e b=20

Após a troca

a=10 e b=20



CHAMADA POR REFERÊNCIA

- Como C passa argumento para funções usando **chamada por valor**, a função chamada **não** pode alterar diretamente uma variável na função chamadora.
- Por causa da chamada por valor, `troca( )` **não** pode afetar os argumentos `a` e `b` na rotina que chama. Na verdade, como é feita uma cópia do valor do argumento para a nova variável, só esta é alterada, ou seja, troca-se apenas `x` e `y`.
- Felizmente, há uma maneira de obter o efeito desejado. O programa chamador passa os endereços dos valores a serem permutados: `troca( &a, &b ) ;`
- E na função `troca( )`, os argumentos são declarados como **ponteiros**, e as variáveis atuais são acessadas através deles.

CHAMADA POR REFERÊNCIA

- A **chamada por referência** é implementada utilizando ponteiros, passando o endereço de memória da variável original para a função.
- Logo a função pode acessar e modificar diretamente o valor da variável original através do seu endereço.
- A chamada por referência é usada quando é necessário que a função altere o valor de uma variável que foi passada para ela.
- Para que uma função gere o efeito de chamada por referência, **ponteiros** devem ser usados na declaração da lista de argumentos e a função chamadora deve mandar endereços como argumentos.

CHAMADA POR VALOR

- Exemplo:

```
int main( )
{
    int  a, b;
    a=10, b=20;
    troca(a , b);
}

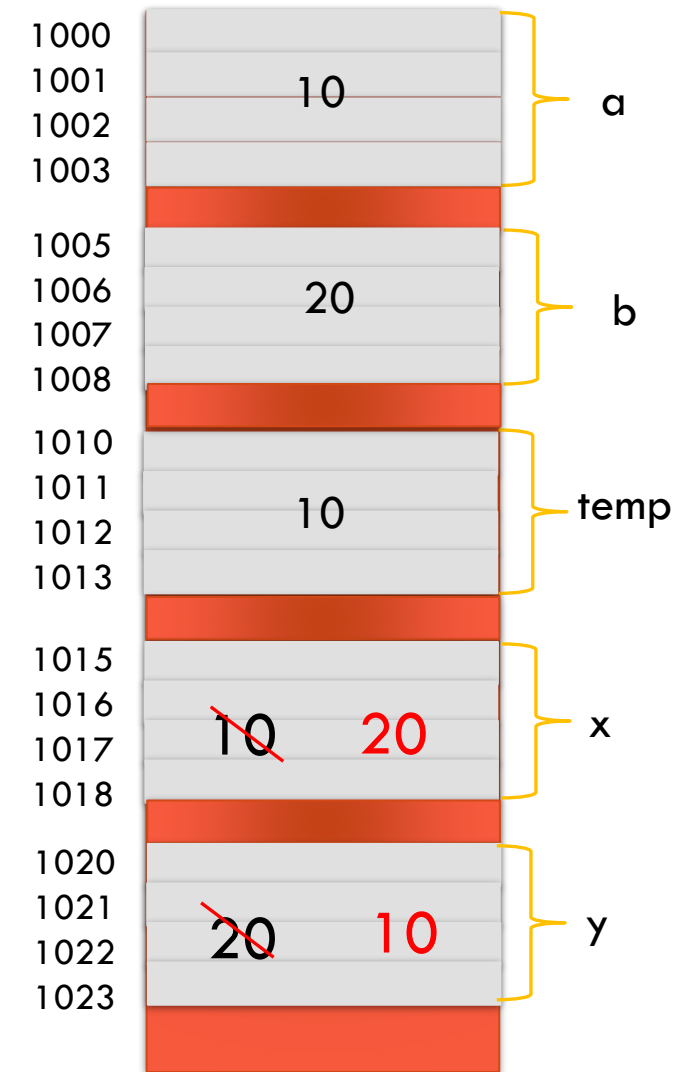
void  troca(int x, int y)
{
    int  temp;
    temp = x;
    x = y;
    y = temp;
}
```

Entrada:

a=10 e b=20

Após a troca

a=10 e b=20



CHAMADA POR REFERÊNCIA

- Exemplo:

```
int main( )
{
    int  a, b;
    a=10, b=20;
    troca(&a , &b);
}

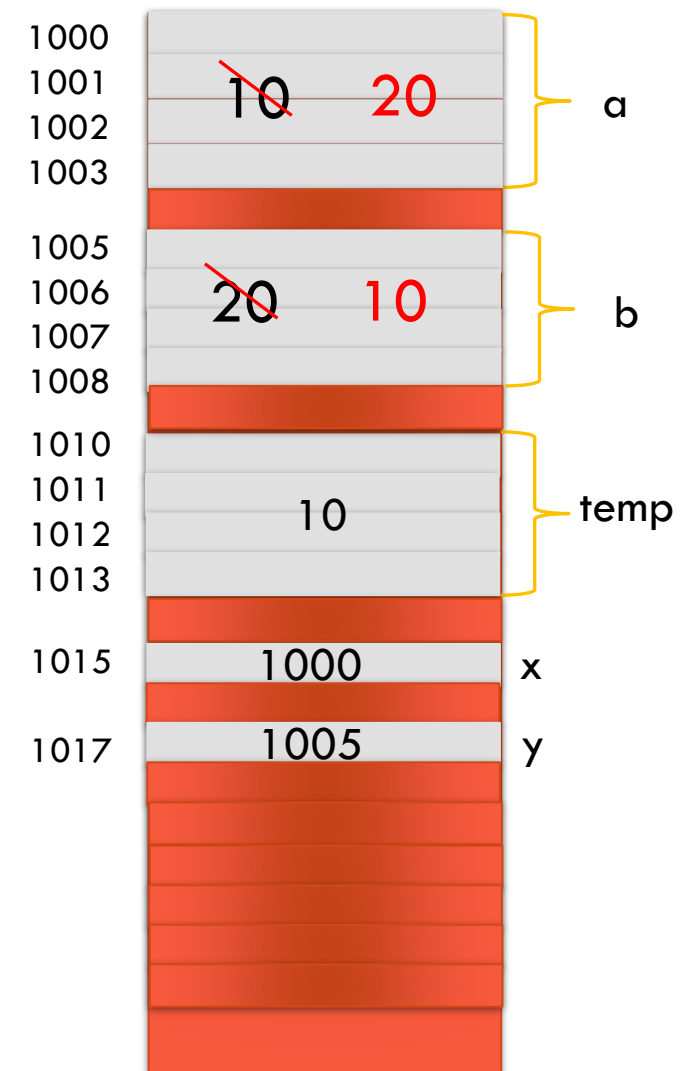
void troca(int *x, int *y)
{
    int  temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

Entrada:

a=10 e b=20

Após a troca

a=20 e b=10



PONTEIROS PARA ACESSAR VETORES

- Exemplo: Com função

```
void calcula_dobro(int *n);  
int main()  
{  
    End. de x é guardado no *n  
    da função  
    int x = 5;  
    calcula_dobro(&x);  
    printf("Dobro = %i", x);  
  
    return 0;  
}
```

```
void calcula_dobro(int *n) {  
    *n *= 2;  
}
```

O compilador compreende
quando * se refere a
multiplicação ou a um ponteiro.

PONTEIROS PARA ACESSAR VETORES

- Exemplo: Sem função

```
int main()
{
    int i, *p, v[5];
    p= v; //inicialização do ponteiro

    for(i=0;i<5;i++,p++)
        scanf("%i",p);
    p=v; // reinicializando
    for(i=0;i<5;i++,p++)
        printf("\nValor: %i",*p);
}
```

É necessário para posicionar o ponteiro novamente para o início do vetor

PONTEIROS PARA ACESSAR VETORES

- Exemplo: Com função

```
void recebe(int *p, int q);  
void mostra(int *p, int q);  
int main()  
{  
    int v[5];  
    recebe(v, 5);  
    mostra(v, 5);  
  
    return 0;  
}
```

End. do vetor é guardado no *p da função



```
void recebe(int *p, int q){  
    int i;  
    for(i=0; i<5; i++, p++)  
        scanf("%i", p);  
}  
void mostra(int *p, int q){  
    int i;  
    for(i=0; i<5; i++, p++)  
        printf("\nValor: %i", *p);  
}
```

PONTEIROS PARA ACESSAR STRUCTS

- Utiliza-se **ponteiros** para guardar endereço de variáveis do tipo da struct e para acessar os membros da struct utiliza-se o operador “->” chamado de **operador de acesso indireto**.
- Exemplo:

```
struct Aluno {int id; float nota;}; //definição struct  
  
//Na main()  
  
struct Aluno a = {1, 9.5};  
struct Aluno *p = &a;  
printf("%.2f", p->nota);
```

PONTEIROS PARA ACESSAR STRUCTS

```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
}aluno;
```

- **Exemplo: Sem ponteiros**

```
void imprime(aluno aux);  
aluno cadastro(int i);  
int main()  
{  
    aluno turma[3];  
    int i;  
  
    for(i=0;i<3;i++){  
        turma[i]=cadastro(i+1);  
    }  
  
    for(i=0;i<3;i++){  
        imprime(turma[i]);  
    }  
    return 0;  
}
```

 **Acessando 3 elementos**

```
aluno cadastro(int i)  
{  
    aluno aux;  
    printf("\nMatr %i :",i);  
    scanf("%i",&aux.matricula);  
    fflush(stdin);  
    printf("Nome %i: ",i);  
    gets(aux.nome);  
    printf("Nota %i: ",i);  
    scanf("%f",&aux.nota);  
    return aux;  
}  
  
void imprime(aluno aux)  
{  
    printf("%i ",aux.matricula);  
    printf("%s ",aux.nome);  
    printf("%f\n",aux.nota);  
}
```

PONTEIROS PARA ACESSAR STRUCTS

- Exemplo: Utilizando Ponteiro

```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
}aluno;  
void imprime(aluno *p);  
void cadastro(aluno *p);
```

```
int main()  
{  
    aluno turma[3];  
    cadastro(turma);  
    imprime(turma);  
    return 0;  
}
```

Guardando
endereço de turma



```
void cadastro(aluno *p){  
    int i;  
    for(i=0;i<3;i++){  
        printf("\nMatr %i :",i+1);  
        scanf("%i",&(p+i)->matricula);  
        fflush(stdin);  
        printf("Nome %i: ",i);  
        gets((p+i)->nome);  
        printf("Nota %i: ",i);  
        scanf("%f",&(p+i)->nota);  
    }  
}  
void imprime(aluno *p){  
    int i;  
    for(i=0;i<3;i++){  
        printf("%i ",(p+i)->matricula);  
        printf("%s ",(p+i)->nome);  
        printf("%f\n", (p+i)->nota);  
    }  
}
```

MODULARIZAÇÃO

- Utilizar subrotinas deve ser então uma prática comum para o programador, mas alguns critérios devem ser adotados para se avaliar a **qualidade** dos diversos módulos em que o programa for dividido.
- Os dois critérios de maior relevância são a **coesão** e o **acoplamento** verificado entre os módulos, que podem ser utilizados tanto na análise da qualidade de subrotinas comuns criadas pela **programação estruturada** como na análise da qualidade da estrutura das classes e métodos especificados na **abordagem orientada a objetos**.

MODULARIZAÇÃO

- O **acoplamento** indica o quanto dois módulos estão interligados entre si.
- Sempre haverá algum acoplamento entre as rotinas de um programa, afinal elas fazem parte de um sistema.
- A questão é decidir quanto de acoplamento devemos permitir, já que uma interligação muito forte entre dois módulos produz problemas na manutenção do programa, devido principalmente ao chamado 'efeito de propagação', em que uma modificação em algum aspecto de uma rotina leva a efeitos colaterais em outra, que por sua vez afetará uma terceira.
- Logo, estamos interessados então em módulos com **baixo acoplamento**.

MODULARIZAÇÃO

- A **coesão** indica o quanto as diversas partes de um módulo estão interligadas, ou seja, em que grau o código de um módulo está concentrado em atingir seu objetivo.
- **Rotinas coesas** são aquelas em que todas as instruções contribuem especificamente para a realização do objetivo declarado da rotina.
- **Rotinas pouco coesas** são aquelas em que as diversas partes do código estão pouco integradas, possivelmente em uma rotina que tenta fazer várias coisas diferentes, como ocorre com o programa todo na função `main()`.
- O projetista de software deve buscar sempre módulos com **coesão elevada**, que possuem um objetivo claro e específico, onde todos os elementos contribuem diretamente para essa finalidade declarada.

CONCLUSÕES

- As **subrotinas** organizam o código de forma a evitar a repetição de código e identificar hierarquias na lógica.
- Existem sub-rotinas chamadas de **funções** e outras de **procedimentos**.
- As funções tem parâmetros e argumentos.
- As funções podem ter passagem por valor ou por referência.
- Os **ponteiros** são usados no caso de passagem por referência.
- Buscamos por programas com **alta coesão** e **baixo acoplamento**.

DÚVIDAS

- Questionamentos?

